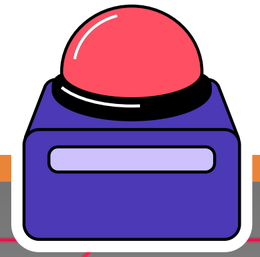
SGU INFORMATION TECHNOLOGY

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING

ALGORITHMS & OBJECT-ORIENTED PROGRAMMING
WEEK 10 LAB

JOSEPH ANDREAS



AGENDA

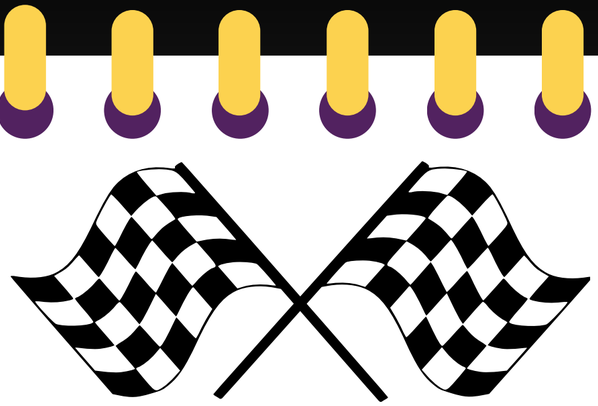
1. OBJECT-ORIENTED PROGRAMMING

2. CLASSES AND OBJECTS

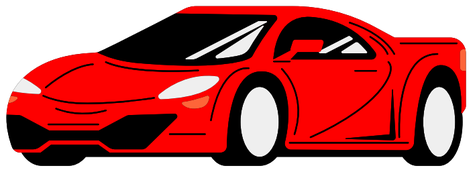
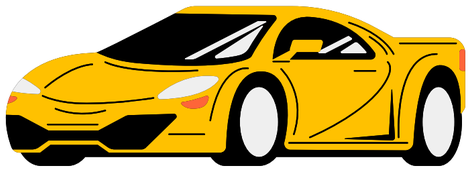
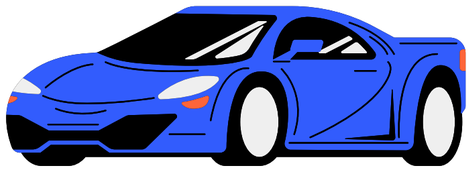
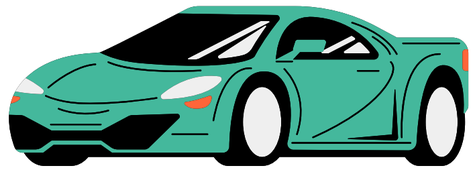
3. CONSTRUCTOR

4. `THIS`

START



OBJECT-ORIENTED PROGRAMMING

WHAT IS OBJECT-ORIENTED PROGAMMING ?

- A programming paradigm
- Based on the concept of "objects"
- Contain data (attributes) and methods (functions) that operate on the data
- Enables modular, reusable, and maintainable code

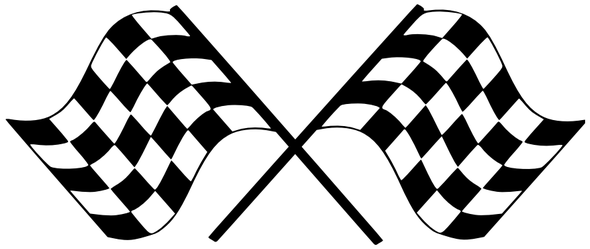
BENEFITS OF OOP

1. Modularity and Code Organization: Code is more organized and easier to manage

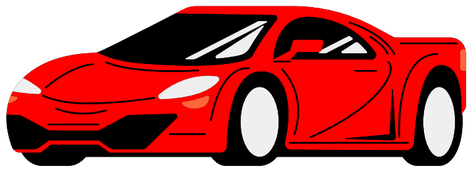
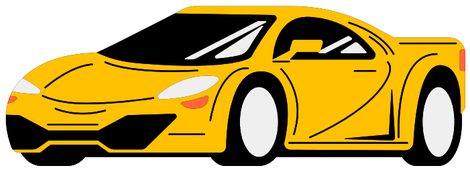
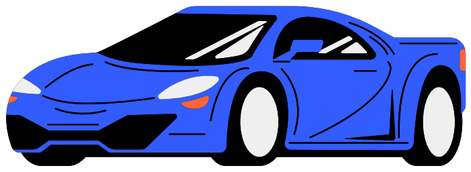
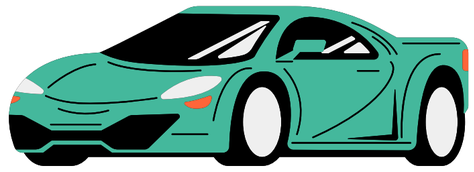
2. Code Reusability: Allows some parts of the code to be reused

3. Scalability: Development of large and complex systems are facilitated by creating code that is easily extendable

4. Enhanced Security: Use of access modifiers (e.g: private, protected) helps in enforcing strict control over how data is accessed and modified



CLASSES AND OBJECTS

WHAT ARE CLASSES AND OBJECTS ?

- Classes: Blueprint for objects. Defines the structure (attributes) and behavior (methods). Created with the `class` keyword

- Objects: Instances of classes. Created from a class

EXAMPLE OF CLASSES AND OBJECTS

```
class Car {
    String color;
    String model;

    void startEngine() {
        System.out.println("Engine started");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.color = "Red";
        myCar.model = "Tesla Model S";
        myCar.startEngine();
    }
}
```

Car is the class

color and model are the attributes

startEngine is the method

myCar is the object

EXAMPLE OF CLASSES AND OBJECTS

```
class Car {
    String color;
    String model;

    void startEngine() {
        System.out.println("Engine started");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.color = "Red";
        myCar.model = "Tesla Model S";
        myCar.startEngine();
    }
}
```

Now, in order to instantiate an object, why does the class name have to be mentioned twice ?

EXAMPLE OF CLASSES AND OBJECTS

```
class Car {
    String color;
    String model;

    void startEngine() {
        System.out.println("Engine started");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.color = "Red";
        myCar.model = "Tesla Model S";
        myCar.startEngine();
    }
}
```

Now, in order to instantiate an object, why does the class name have to be mentioned twice ?

First occurence (Car myCar): Declare a variable named `myCar` that will hold a reference to an object of type `Car`

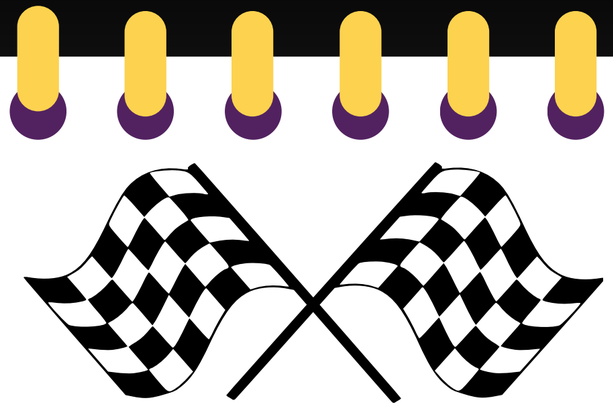
Second occurence (new Car()): Create a new instance (object) of the `Car` class

USING MULTIPLE CLASSES

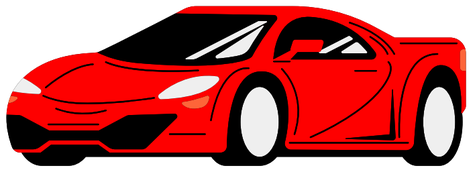
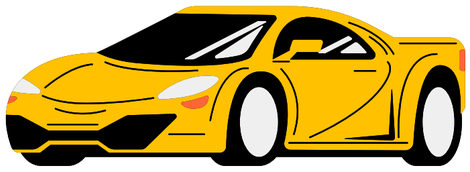
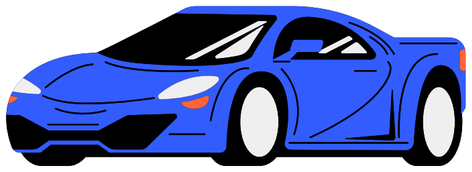
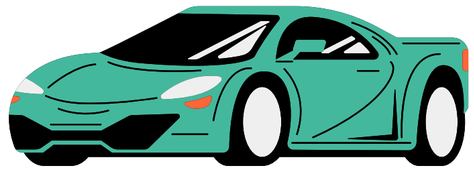
```
// Main.java

public class Main {
  int x = 5;
}
```

```
// Second.java

class Second {
  public static void main(String[] args) {
    Main myObj = new Main();
    System.out.println(myObj.x);
  }
}
```

Possible to create an object of class and access it in another class. Often used for better class organization



CONSTRUCTOR

WHAT IS CONSTRUCTOR ?

- Special method used to initialize objects
- Called when an instance (or object) of a class is created
- Sets up the initial state of the object, typically by assigning values to its fields (or attributes)

CHARACTERISTICS OF A CONSTRUCTOR

- Same Name as Class: A constructor must have the same name as the class in which it is defined
- No Return Type: Unlike regular methods, constructors don't have a return type, not even void
- Automatic Call: When an object is created using the new keyword, the constructor is called automatically
- Types of Constructors:
  - Default Constructor: A no-argument constructor that initializes fields with default values (e.g., null for objects, 0 for integers)
  - Parameterized Constructor: A constructor with parameters that allows initializing objects with specific values

CONSTRUCTORS

```
public class Book {
    String title;
    String author;

    public Book() {
        this.title = "Unknown";
        this.author = "Unknown";
    }

    public Book(String title, String author) {
        this.title = title;
        this.author = author;
    }

    public void displayBookDetails() {
        System.out.println("Title: " + this.title);
        System.out.println("Author: " + this.author);
    }
```

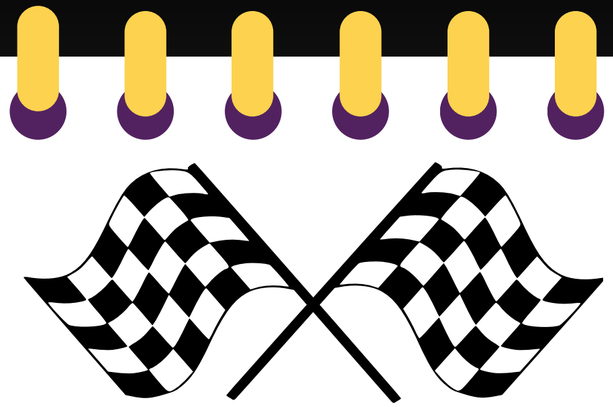
Default Constructor

Parameterized Constructor

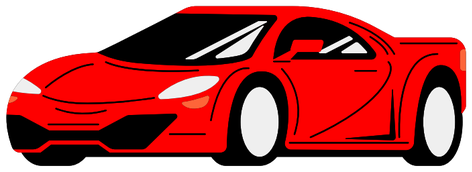
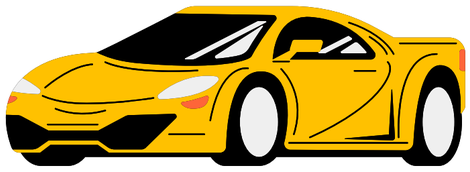
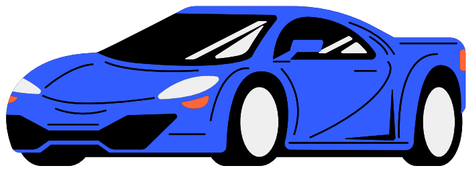
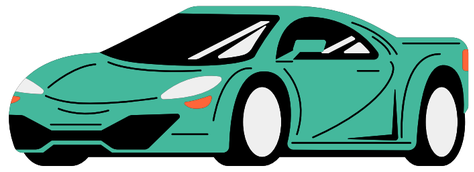
```
public static void main(String[] args) {
        Book book1 = new Book();
        Book book2 = new Book("Depressing Emotions", "Tom Cruise");
        book1.displayBookDetails();
        book2.displayBookDetails();
    }
}
```

book1 will output Unknown as no values are provided

book2 will output the set title and author



'THIS'

WHAT IS `THIS`?

- Used in Java to refer to the current instance of the class

- Uses:
  - Distinguishing Between Variables: When a parameter in a method or constructor has the same name as a class field, `this` helps to refer specifically to the field of the current object
  - Calling Another Constructor: `this()` can be used to call one constructor from another constructor within the same class, allowing for code reuse
  - Passing the Current Object: `this` can be used to pass the current object to other methods or constructors
  - Returning the Current Object: `this` can also be used to return the instance itself from a method

'THIS' IN CONSTRUCTOR

```
public class Book {
    String title;
    String author;

     public Book() {
       this.title = "Unknown";
       this.author = "Unknown";
    }

      public Book(String title, String author) {
       this.title = title;
       this.author = author;
    }

    public void displayBookDetails() {
        System.out.println("Title: " + this.title);
        System.out.println("Author: " + this.author);
    }
```

this.title and this.author refers to the title and author defined in the class

'THIS' TO PASS CURRENT OBJECT

```
class Person {
    private String name;

    public Person(String name) {
        this.name = name;
    }

    public void befriend(Person otherPerson) {
        System.out.println(this.name + " is now
friends with " + otherPerson.name);
    }

    public void makeFriendWith(Person
friend) {
        friend.befriend(this);
    }
}
```

```
public class PersonMain {
    public static void main(String[] args) {
        Person person1 = new Person("Alice");
        Person person2 = new Person("Bob");

        person1.makeFriendWith(person2);
    }
}
```

1. Method Call:

person1.makeFriendWith(person2);

- person1 is calling the makeFriendWith method, and it passes person2 as an argument

- Inside makeFriendWith, friend refers to person2

'THIS' TO PASS CURRENT OBJECT

```
class Person {
    private String name;

    public Person(String name) {
        this.name = name;
    }

    public void befriend(Person otherPerson) {
        System.out.println(this.name + " is now
friends with " + otherPerson.name);
    }

    public void makeFriendWith(Person
friend) {
        friend.befriend(this);
    }
}
```

```
public class PersonMain {
    public static void main(String[] args) {
        Person person1 = new Person("Alice");
        Person person2 = new Person("Bob");

        person1.makeFriendWith(person2);
    }
}
```

2. Inside makeFriendWith:
   - In this method, this refers to the object that called makeFriendWith. In this case, person1 called it, so this refers to person1
   - When friend.befriend(this); is executed:
     - friend refers to person2
     - this refers to person1
   - The line friend.befriend(this); translates to person2.befriend(person1)

'THIS' TO PASS CURRENT OBJECT

```
class Person {
    private String name;

    public Person(String name) {
        this.name = name;
    }

    public void befriend(Person otherPerson) {
        System.out.println(this.name + " is now
friends with " + otherPerson.name);
    }

    public void makeFriendWith(Person
friend) {
        friend.befriend(this);
    }
}
```

```
public class PersonMain {
    public static void main(String[] args) {
        Person person1 = new Person("Alice");
        Person person2 = new Person("Bob");

        person1.makeFriendWith(person2);
    }
}
```

3. Calling befriend:
   - person2.befriend(person1); is executed, originated from friend.befriend(this)
   - Inside befriend, this now refers to person2 (since befriend was called on person2), and otherPerson is person1

`THIS` TO PASS CURRENT OBJECT

```
class Person {
    private String name;

    public Person(String name) {
        this.name = name;
    }

    public void befriend(Person otherPerson) {
        System.out.println(this.name + " is now
friends with " + otherPerson.name);
    }

    public void makeFriendWith(Person
friend) {
        friend.befriend(this);
    }
}
```

```
public class PersonMain {
    public static void main(String[] args) {
        Person person1 = new Person("Alice");
        Person person2 = new Person("Bob");

        person1.makeFriendWith(person2);
    }
}
```

4. Final Output:
   - Inside befriend, this.name is person2.name (which is "Bob")
   - otherPerson.name is person1.name (which is "Alice")

`THIS` TO RETURN CURRENT OBJECT

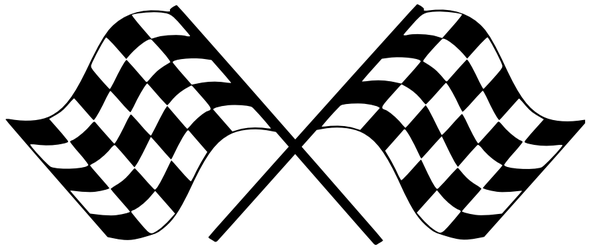
```
class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getDescription() {
        return name + " is " + age + " years old.";
    }

    public Person getCurrentPerson() {
        return this;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Person person1 = new Person("Alice",
30);

        Person currentPerson =
person1.getCurrentPerson();

System.out.println(currentPerson.getDescri
ption());
    }
}
```

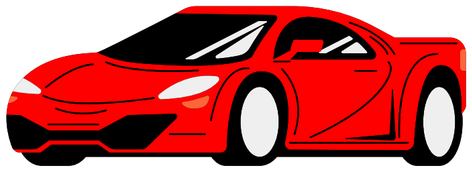
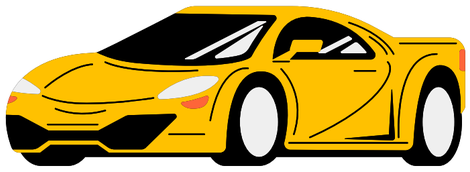
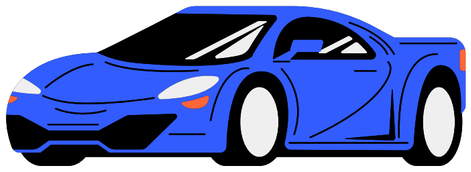
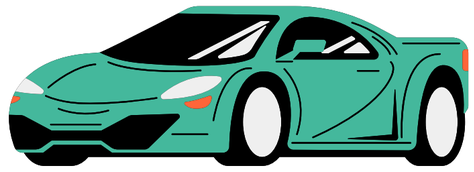
1. Creating the Person: person1 is created with the name "Alice" and age 30

2. Getting the Current Instance: When person1.getCurrentPerson() is called, it returns `this`, which refers to person1 itself. This instance is assigned to currentPerson

3. currentPerson.getDescription() will result in “Alice is 30 years old”



ANY QUESTION?

SGU INFORMATION TECHNOLOGY

THANK YOU FOR YOUR ATTENTION!